COMP90086
Assignment 2

Yu Luo
Student ID: 1528667

September 12, 2025

1 The Image Recognition Task

This image recognition task is challenging due to several factors related to the images' composition and the characteristics of the target objects. The task involves recognising a small square patch containing a letter (T or L) embedded in a noisy background of random rectangles.

1.1 Challenges From Computer Vision Perspective

- **Positional and Scale Variation:** The letter patch can appear **anywhere** in the 28×28 grayscale image and its size can vary, being 5, 7, or 9 pixels square. This requires the model to be robust to translation and scale changes.
- **Background Clutter:** The background is composed of random rectangles that can share basic visual features like edges and corners with the target letters. This clutter can make it difficult for a model to isolate and identify the letter itself.
- **Within-Class Variation:** Although the letters are always brighter than their immediate background patch, the overall brightness of the image and the rectangles can vary, meaning the model cannot rely on a fixed intensity threshold. The classifier must learn to ignore these variations, along with the different positions and sizes of the letters.
- **Truncation at borders:** Letters located near the image edge can have strokes clipped, removing critical cues. For example, a `T` with its horizontal bar cut off may resemble an `L`, leading to misclassification.

1.2 Features to Detect

To correctly classify the images, the model must focus on structural properties that distinguish a `T` from an `L` despite noise and variability.

- **T-Shape (T-Junction):** Defined by a vertical stroke intersecting the midpoint of a horizontal bar, forming a clear junction.
- **L-Shape (Corner):** Characterised by an isolated right angle where two perpendicular strokes meet at their endpoints, without a central intersection.
- **Local Brightness Contrast:** Letter strokes are consistently brighter than the immediate background, so the model must exploit relative rather than absolute intensity differences.

- **Geometric Complexity:** A useful separator is the number of vertices—an `L` typically forms three, while a `T` has four due to its junction. This structural cue helps avoid confusion between the two shapes.

1.3 Irrelevant Variations to Ignore

The classifier must remain invariant to factors that do not define class identity, including:

- **Position:** the location of the letter within the 28×28 grid;
- **Scale:** the size of the patch, whether 5×5, 7×7, or 9×9;
- **Background Clutter:** the arrangement and brightness of random rectangles in the background;
- **Clipped Strokes:** partial truncation at the image borders or mild pixel-level noise.

These within-class variations must be ignored so that the classifier focuses only on the discriminative structures distinguishing a `T` from an `L`.

1.4 Hand-designed Convolutional Filters

The most effective approach is to target discriminative structures such as edges, corners, and junctions. Since letter strokes occupy only a few pixels, small kernels (e.g. 3×3) are well suited to capture these local patterns. Larger kernels (e.g. 5×5 or 7×7) can also be used to capture slightly broader context in a single step. To achieve robustness, the filters should be rotated and mirrored to cover all possible orientations of `T` and `L` shapes.

Horizontal Edge Detector

-1	-1	-1
0	0	0
1	1	1

Vertical Edge Detector

1	0	-1
1	0	-1
1	0	-1

Figure 1: Basic 3×3 Sobel-style filters for horizontal and vertical edge detection.

Corner Detector (bottom-left)

1	-1	-1
1	-1	-1
1	1	1

T-Junction Detector

1	1	1
-1	1	-1
-1	1	-1

Figure 2: Filters designed to detect an `L`-like corner and a `T`-junction.

These example kernels illustrate the type of local structures a CNN would learn automatically. Edge filters capture stroke orientation, the corner filter responds to `L`-shaped intersections, and the T-junction filter activates for the defining structure of a `T`. A complete filter bank with rotated variants ensures coverage across positions and orientations.

2 CNN Architecture

2.1 Experiments

Setup. In my experiments, I varied network depth, filter width, kernel size, and padding to balance accuracy with parameter efficiency. The goal was to maximise validation/test accuracy while keeping the model compact. A fixed random seed (42) ensured reproducibility. For Experiments 1 and 2, all models used ReLU activation and SAME padding, differing only in convolutional kernel size and layer configuration. All the experiments are conducted within a training time of 20 epochs.

Experiment Results. Table 1 summarises the final validation accuracy and parameter counts for all tested model configurations. Figures 3–5 show the corresponding training and validation accuracy curves.

Architecture	Parameters	Validation Accuracy
1conv, 16f, 7×7	201,633	0.6104
1conv, 16f, 5×5	201,249	0.5413
1conv, 16f, 3×3	200,993	0.5477
2conv, 16f–16f, 3×3	52,785	0.7846
2conv, 16f–32f, 3×3	105,281	0.8089
2conv, 32f–32f, 3×3	110,049	0.7982
2conv, 32f–64f, 3×3	219,649	0.9180
2conv, 64f–64f, 3×3	238,401	0.8292
2conv, 32f–64f, 5×5 → 3×3	220,161	**0.9569**
2conv, 32f–64f, 7×7 → 5×5	253,697	0.9390
2conv, 32f–64f, 7×7 → 3×3	220,929	0.9047
2conv, 32f–64f, 5×5 → 3×3 (padding=same)	220,161	0.9569
2conv, 32f–64f, 5×5 → 3×3 (padding=valid)	121,857	0.9488

Table 1: Validation accuracy and parameter counts across model configurations.

2.2 Discussion

Experiment 1 (Depth and Width). As shown in Figure 3, two-layer CNNs consistently outperformed single-layer networks, confirming the importance of hierarchical feature extraction (such as edges → corners/junctions). Accuracy improved with wider layers, especially when the second layer had more filters (32→64, reaching 0.9180). However, the very wide symmetric model (64→64) underperformed (0.8292), suggesting over-capacity and weaker generalisation.

Experiment 2 (Kernel Size Pattern). Figure 4 shows that using a larger first kernel (5×5) followed by a smaller second kernel (3×3) achieved the highest accuracy (0.9569). In contrast, oversized kernels (7×7) performed worse and required longer training to reach comparable accuracy (green curve), likely because they smoothed out fine local details essential for distinguishing `T` from `L`. This confirms that combining a moderately large receptive field with a smaller local detector is most effective.

Experiment 3 (Padding). As shown in Figure 5, SAME padding achieved slightly higher accuracy (0.9569) than VALID (0.9488), but at the cost of nearly doubling the parameter count

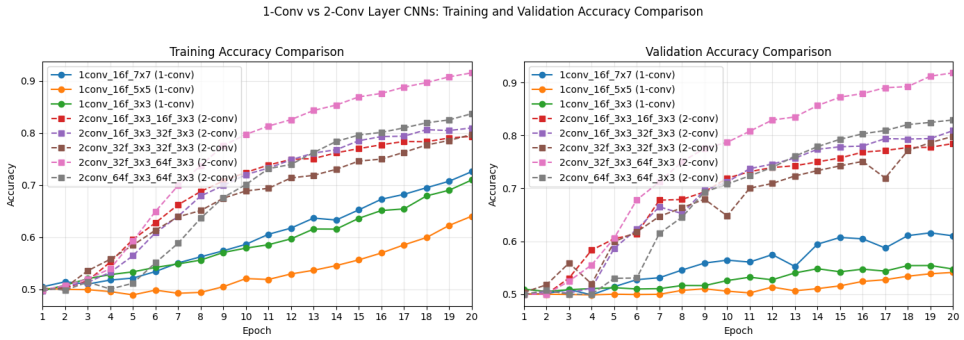

Figure 3: Learning curves for depth/width experiments (Experiment 1).

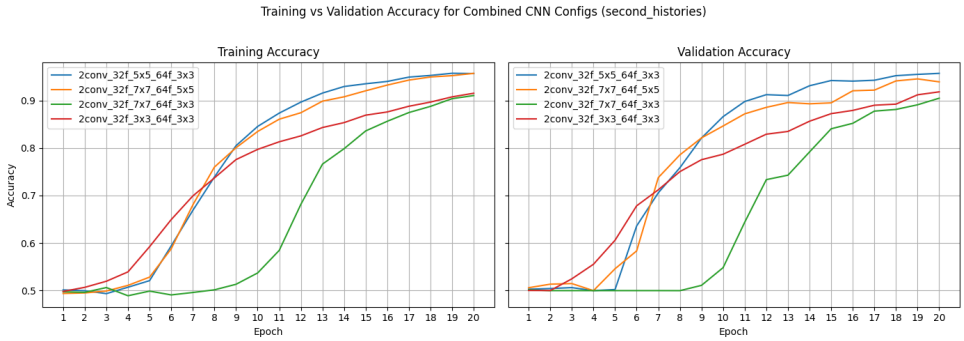

Figure 4: Learning curves for kernel size experiments (Experiment 2).

(220k vs. 122k). VALID padding may converge more slowly due to border cutoffs, yet it offers a better trade-off by retaining most of the accuracy while substantially reducing model size.

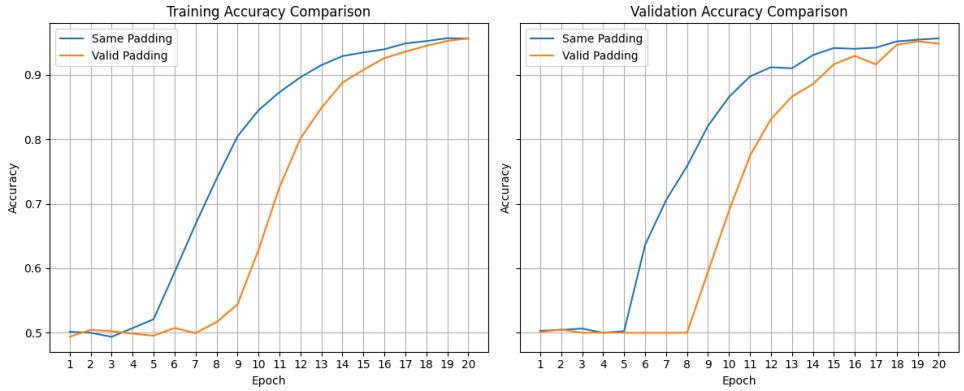

Figure 5: Learning curves for padding experiments (Experiment 3).

2.3 Final Model

Based on these results, the final chosen model is shown in Figure 6. A Dropout layer with rate 0.2 was added to reduce overfitting, and the model was trained for 30 epochs to ensure stable convergence. The final model achieved a validation accuracy of **97.86%** with a total parameter count of **121,857** (sees code for detail).

Training and Validation Curves. Figures 7 and 8 present the learning curves for the final model. Accuracy increases steadily and plateaus near 98%, with validation closely tracking

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 28, 28, 1)	0
conv2d_23 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_23 (MaxPooling2D)	(None, 12, 12, 32)	0
conv2d_24 (Conv2D)	(None, 10, 10, 64)	18,496
max_pooling2d_24 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_13 (Flatten)	(None, 1600)	0
dense_26 (Dense)	(None, 64)	102,464
dropout (Dropout)	(None, 64)	0
dense_27 (Dense)	(None, 1)	65

Figure 6: Summary of the final CNN architecture including two convolutional layers, a dense hidden layer, and a dropout of 0.2.

training—evidence of strong generalisation. The loss decreases smoothly overall, with only minor fluctuations, still indicating stable optimisation and a well-regularised model.

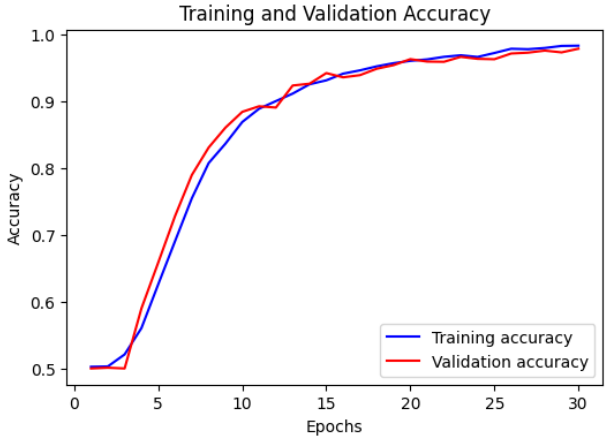

Figure 7: Training and validation accuracy curves for the final model.

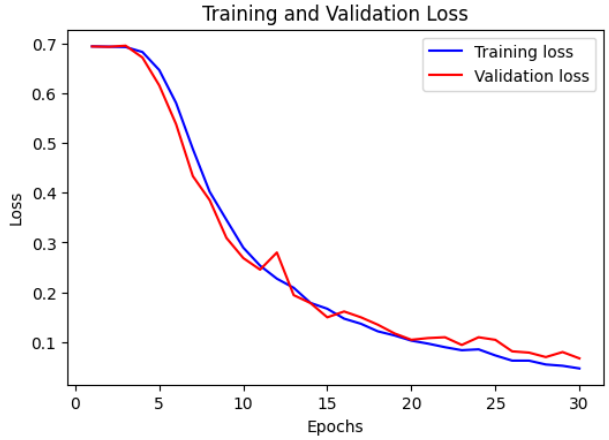

Figure 8: Training and validation loss curves for the final model.

3 Kernel visualisation

3.1 Learned Kernels

The first-layer filters (Figure 9) resemble simple edge and corner detectors, similar to Sobel filters. Horizontal edges are visible in kernels K7, K17, and K31, vertical edges in K23 and K25, and corners in K5, K6, and K8. Some kernels (e.g., K24, K27, K31) highlight junction-like structures, providing the basic primitives for recognising the strokes of `T` and `L`.

The second-layer filters (Figure 10) are more shape-specific, with kernels such as K7, K26, K27, K32, and K40 resembling `L`-corners, while others (e.g., K36, K49, K57) capture T-junctions. These higher-level features align directly with the discriminative structures needed to distinguish `T` from `L`.

From my observation, the model mostly learned the filters expected in Question 1: early layers capture edges and corners, while deeper layers combine them into junctions and intersections. This hierarchical progression confirms that the CNN can focus on the defining structural cues—corners for `L` and T-junctions for `T`—rather than background noise.

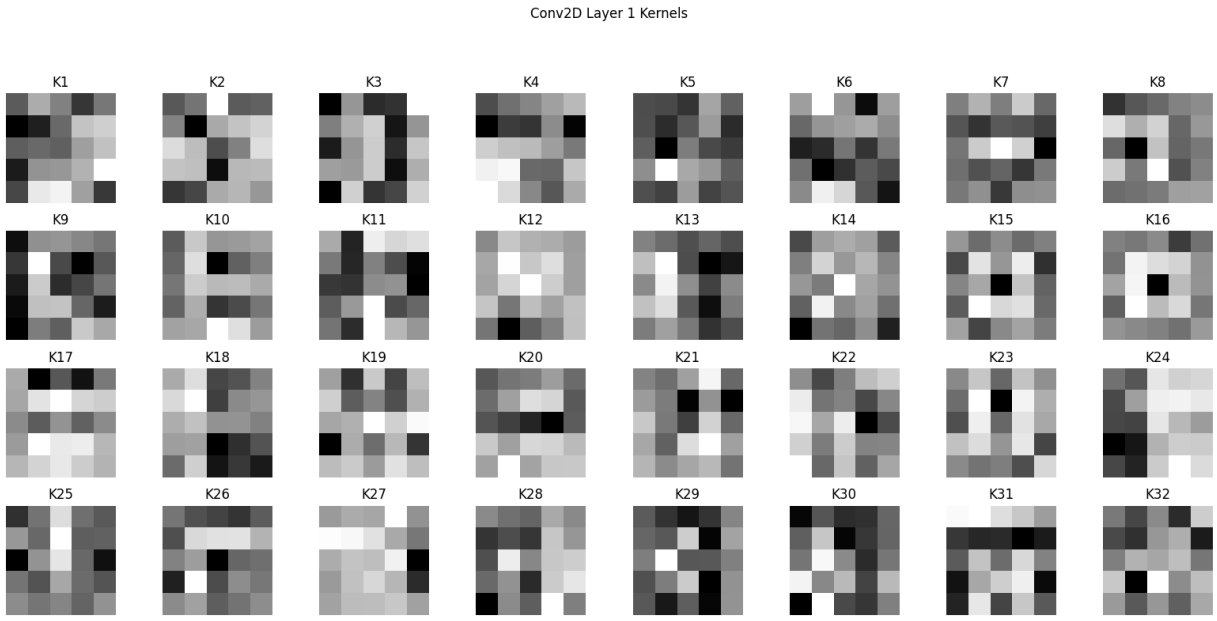

Figure 9: Learned kernels from the first convolutional layer.

4 Where is the Model Looking?

Grad-CAM was applied to randomly selected validation samples for both classes `L` and `T`. Each figure shows the input image alongside the overlaid heatmap, with correctly predicted samples annotated in green and misclassified ones in red.

In Figure 11, most samples were correctly classified as `L`, with only one misclassified as `T`. Notably, even in the misclassified case, the highlighted region does not resemble a true T-junction but instead emphasises a generic corner, suggesting that the model sometimes confuses corner-like features with T-junctions. For the correctly classified `L` samples, attention is often directed not to the actual letter but to nearby areas of strong contrast (e.g., Sample 9 focusing on the bottom-right corner). In some cases (e.g., Samples 2 and 3), the model appears to rely on isolated line segments. This indicates a tendency to classify based on local edges and corners rather than the full global structure of the letter.

In Figure 12, similar behaviour is observed, with activations frequently concentrating on regions of strong contrast (e.g., Samples 9 and 10). However, compared to the `L` class, the model more often attends to the letter itself. For instance, Samples 2, 7, and 8 show the heatmap roughly centred on the intersection of vertical and horizontal strokes, the precise discriminative feature of `T`. An exception is Sample 1, where the model focused on a homogeneous patch, highlighting occasional sensitivity to background noise.

Overall, these results reveal a key asymmetry: when classifying `L`, the model often focus onto spurious corners in the background, which can both help (by reinforcing L-like cues) and hinder (by distracting from the true letter). In contrast, when classifying `T`, the model is more focused on locating the actual T-junction, likely because this feature is harder to detect in cluttered scenes.

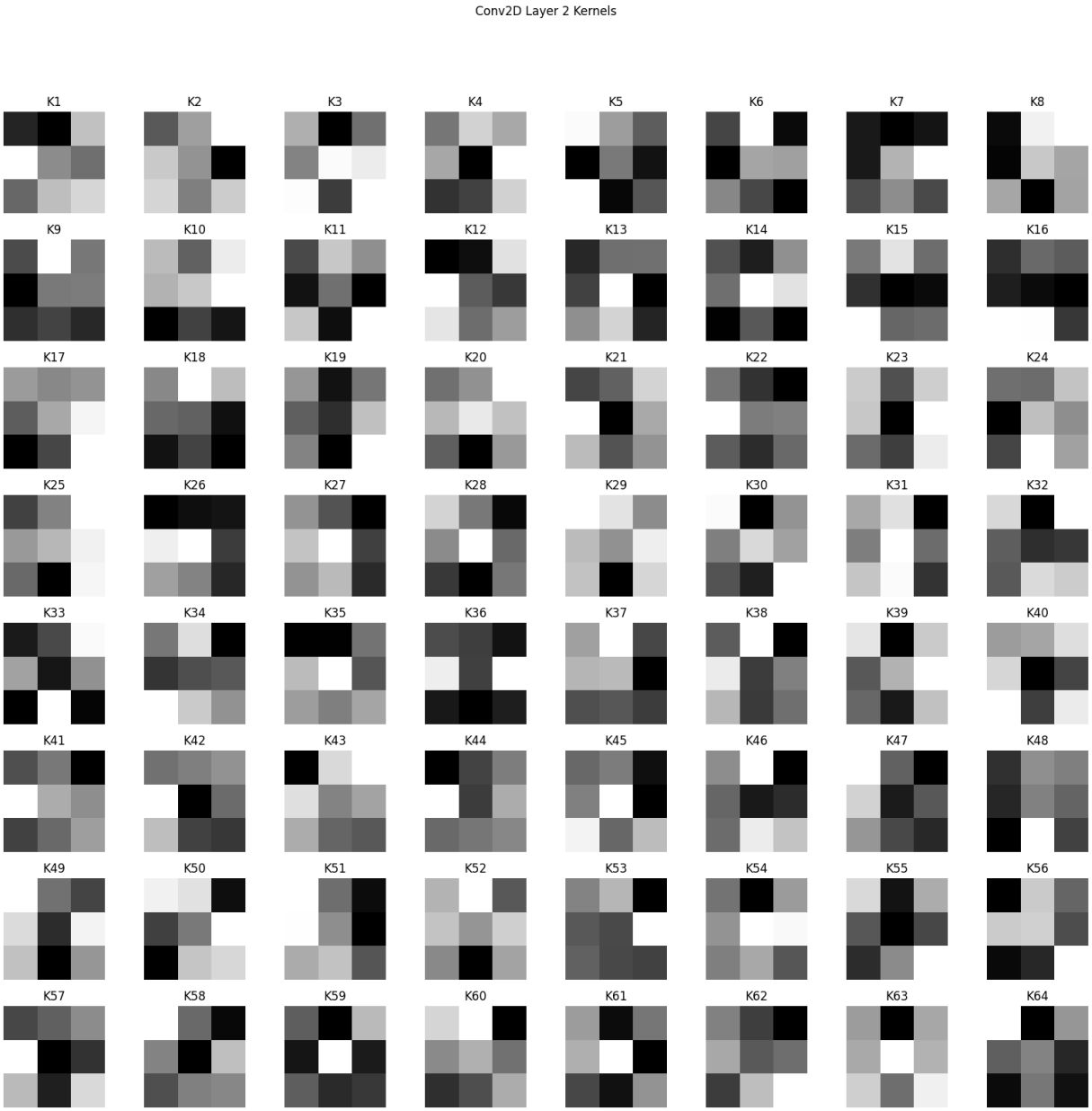

Figure 10: Learned kernels from the second convolutional layer.

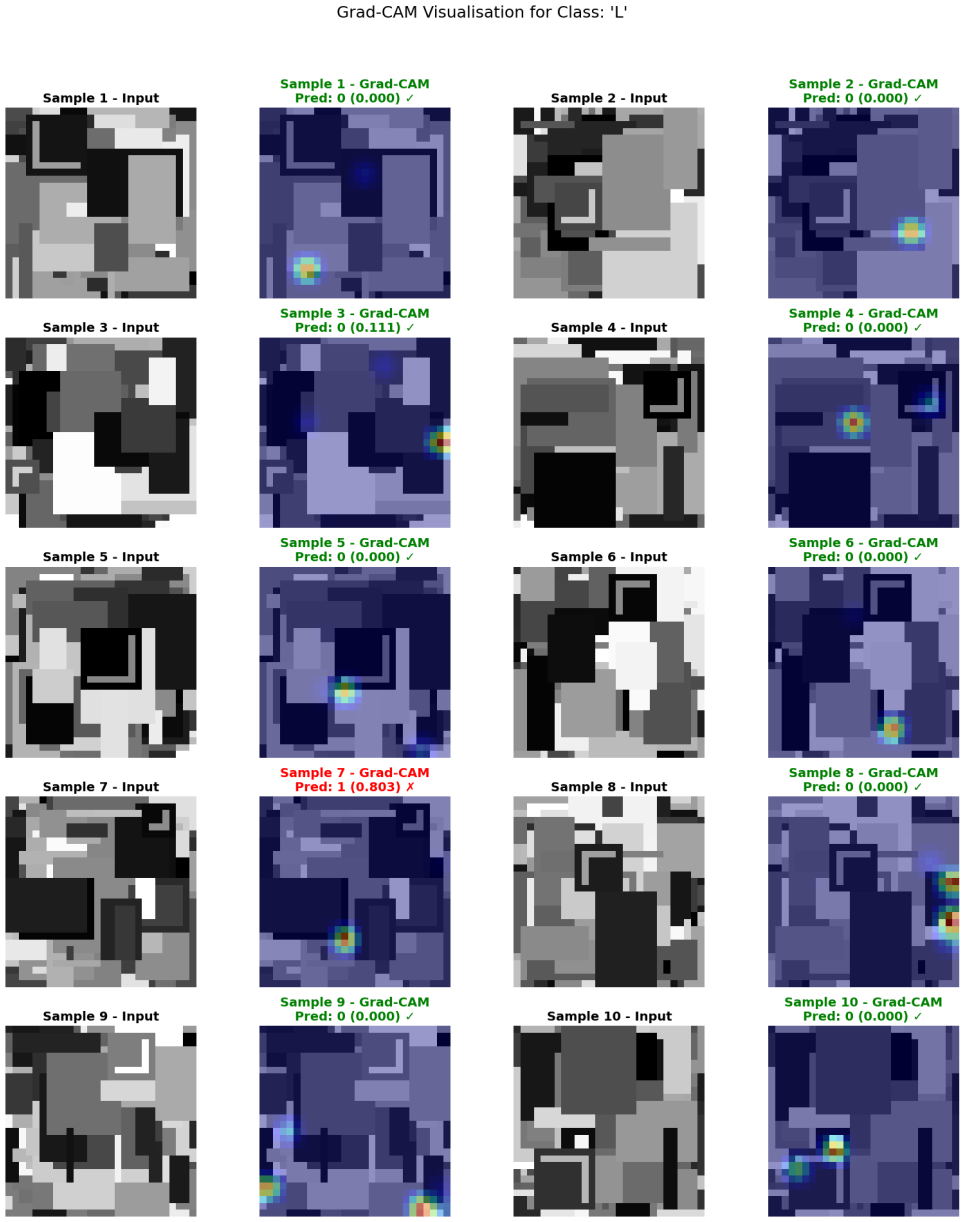

Figure 11: Grad-CAM visualisations for samples classified as L. Most predictions attend to corner-like regions, though not always aligned with the actual letter.

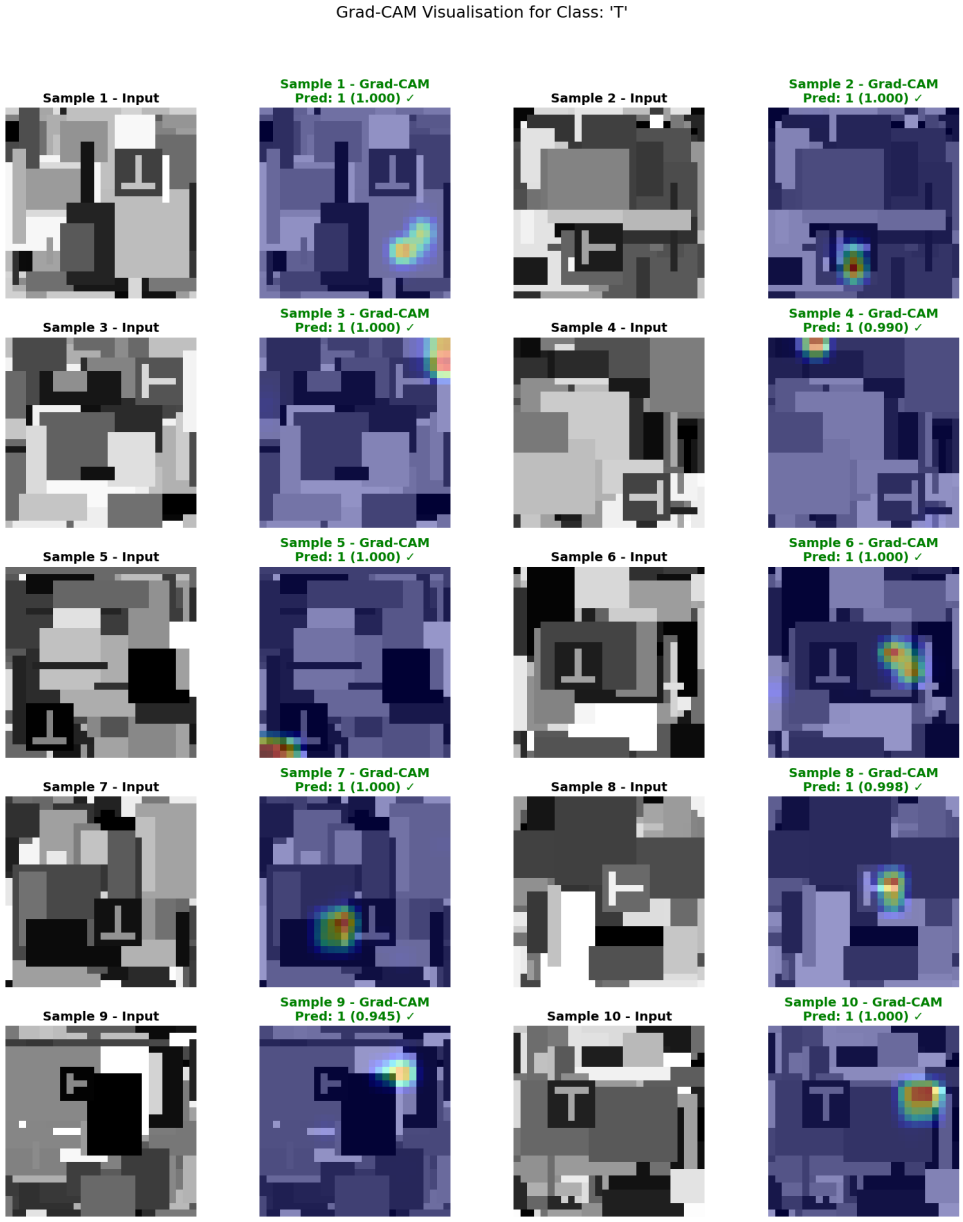

Figure 12: Grad-CAM visualisations for samples classified as `T`. The model more frequently attends to the true T-junction regions, though occasional misfocus occurs.